Deployment

A CEREBRUM Research Publication (v1.2.4)

Bryan Alexander Buchorn
Independent Researcher

Abstract

Overview: This guide provides a conceptual entry point into the CEREBRUM framework. It is designed for practitioners and researchers seeking to understand the core reasoning signals of Framework Implementation Guide.

0.1 CEREBRUM Deployment Guide

Version: v1.1.0 **Audience**: DevOps, Platform Engineers, System Administrators

—

Quick Start (Development)

```
pip install -e ".[api]"
export CEREBRUM_JWT_SECRET="dev-secret-change-in-production"
uvicorn api.server:app --port 8200 --reload
curl http://localhost:8200/health
```

—

Environment Variables

Variable	Required	Default	Description
CEREBRUM_JWT_SECRET	**Yes (prod)**	`dev-insecure-secret`	HMAC-SHA256 key for JWT signing
CEREBRUM_JWT_TTL	No	`3600`	Token lifetime in seconds
CEREBRUM_HMAC_KEY	No	same as JWT_SECRET	Key for path provenance HMAC
CEREBRUM_ALLOW_ANONYMOUS	No	`false`	Disable auth (dev only)
CEREBRUM_AUDIT_LOG	No	`false`	Log query content for compliance
CEREBRUM_GRAPH_CSV	No	—	Path to CSV graph file to auto-load on startup
CEREBRUM_DB_PATH	No	`cerebrum.db`	SQLite persistence file path
CEREBRUM_MAX_HOPS	No	`5`	Global max hops cap
CEREBRUM_BEAM_WIDTH	No	`10`	Default beam width
CEREBRUM_WORKERS	No	`4`	Uvicorn worker count
CEREBRUM_LOG_LEVEL	No	`INFO`	Logging level

—

Docker

Single Container

```
# Dockerfile (already in repo root)
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install -e ".[api]"
EXPOSE 8200
CMD ["uvicorn", "api.server:app", "--host", "0.0.0.0", "--port", "8200", "--
    workers", "4"]
```

```
docker build -t cerebrum:1.1.0 .
docker run -d \
  -p 8200:8200 \
  -e CEREBRUM_JWT_SECRET="$(openssl rand -hex 32)" \
  -e CEREBRUM_GRAPH_CSV=/data/my_graph.csv \
  -v /path/to/data:/data \
  --name cerebrum \
  cerebrum:1.1.0
```

Docker Compose (with Reasoning Studio)

```
# docker-compose.yml
version: "3.9"

services:
  api:
    build: .
    image: cerebrum:1.1.0
    ports:
      - "8200:8200"
    environment:
      CEREBRUM_JWT_SECRET: ${CEREBRUM_JWT_SECRET}
      CEREBRUM_GRAPH_CSV: /data/graph.csv
      CEREBRUM_DB_PATH: /data/cerebrum.db
      CEREBRUM_WORKERS: "4"
    volumes:
      - graph_data:/data
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8200/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  studio:
    build:
      context: .
      dockerfile: ui/Dockerfile
    ports:
```

```
      - "7860:7860"
    environment:
      CEREBRUM_API_URL: http://api:8200
      CEREBRUM_JWT_SECRET: ${CEREBRUM_JWT_SECRET}
    depends_on:
      api:
        condition: service_healthy
    restart: unless-stopped

volumes:
  graph_data:
```

```
# Generate a strong secret
export CEREBRUM_JWT_SECRET="$(openssl rand -hex 32)"
docker compose up -d
```

—

Reverse Proxy (nginx)

```
# /etc/nginx/sites-available/cerebrum
server {
    listen 443 ssl http2;
    server_name cerebrum.yourdomain.com;

    ssl_certificate     /etc/letsencrypt/live/cerebrum.yourdomain.com/
       fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/cerebrum.yourdomain.com/privkey.
       pem;
    ssl_protocols       TLSv1.2 TLSv1.3;

    # API
    location /api/ {
        proxy_pass         http://127.0.0.1:8200/;
        proxy_set_header   Host $host;
        proxy_set_header   X-Real-IP $remote_addr;
        proxy_set_header   X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 30s;

        # Rate limiting
        limit_req zone=cerebrum_api burst=20 nodelay;
    }

    # SSE streams (no timeout)
    location /api/stream/ {
        proxy_pass             http://127.0.0.1:8200/stream/;
        proxy_set_header       Connection '';
        proxy_http_version     1.1;
        chunked_transfer_encoding on;
        proxy_buffering        off;
        proxy_cache            off;
        proxy_read_timeout     3600s;
    }
```

```
    # Reasoning Studio
    location / {
        proxy_pass http://127.0.0.1:7860/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}

# Rate limiting zone
limit_req_zone $binary_remote_addr zone=cerebrum_api:10m rate=100r/m;
```

—

Production Checklist

Security

- CEREBRUM_JWT_SECRET is a random 256-bit hex string (not the dev default)
- CEREBRUM_ALLOW_ANONYMOUS is `false` (default)
- TLS 1.2+ enabled at the reverse proxy layer
- JWT token TTL set to $\leq$ 3600s for high-security environments
- CEREBRUM_AUDIT_LOG=`true` if compliance logging is required

Performance

- CEREBRUM_WORKERS set to `2 × CPU_cores + 1` for CPU-bound workloads
- CEREBRUM_BEAM_WIDTH tuned for your graph size (see `docs/PERFORMANCE_TUNING.md`)
- SQLite persistence file on SSD storage
- For graphs >500K edges: consider in-memory adapter (disable SQLite persistence)

Reliability

- Health check endpoint monitored: `GET /health`
- Docker restart policy: `unless-stopped`
- Log rotation configured for uvicorn access logs
- Data volume backed up (SQLite DB + graph CSV)

—

Neo4j Backend Deployment

```
pip install -e ".[neo4j]"
```

```
from adapters.neo4j_adapter import Neo4jAdapter

adapter = Neo4jAdapter(
    uri=os.environ["NEO4J_URI"],          # bolt://neo4j:7687
    user=os.environ["NEO4J_USER"],        # neo4j
    password=os.environ["NEO4J_PASSWORD"],
)
```

> Neo4j credentials must be provided via environment variables, never hardcoded.
> —

Kubernetes (Helm Values Reference)

```
# values.yaml
replicaCount: 2

image:
  repository: cerebrum
  tag: "1.1.0"
  pullPolicy: IfNotPresent

env:
  CEREBRUM_JWT_SECRET:
    secretKeyRef:
      name: cerebrum-secrets
      key: jwt-secret
  CEREBRUM_WORKERS: "4"
  CEREBRUM_DB_PATH: /data/cerebrum.db

persistence:
  enabled: true
  size: 10Gi
  storageClass: standard

service:
  type: ClusterIP
  port: 8200

ingress:
  enabled: true
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
  hosts:
    - host: cerebrum.yourdomain.com
      paths: ["/"]
  tls:
    - secretName: cerebrum-tls
      hosts: ["cerebrum.yourdomain.com"]
```

```
resources:
  requests:
    cpu: 500m
    memory: 512Mi
  limits:
    cpu: 2000m
    memory: 2Gi

livenessProbe:
  httpGet:
    path: /health
    port: 8200
  initialDelaySeconds: 10
  periodSeconds: 30
```

—

Monitoring

The `/admin/resource-stats` endpoint (requires `admin` scope JWT) returns current CPU, memory, active query count, and rebalance metrics. Integrate with Prometheus via a scrape config targeting this endpoint.

Key metrics to alert on:

- `active_queries` >50 — consider scaling
- `queued_events` >5,000 — stream ingest falling behind
- `rebalance_count` growing rapidly — graph instability, check `q_drift_threshold`